University
Mohammed VI
Polytechnic

College of
Computing

Lab 4: Functional Dependencies

Data Management Course

UM6P College of Computing

Professor: Karima Echihabi **Program:** Computer Engineering
Session: Fall 2026

Team Information

Member 1	BENYASSINE Rayane
Member 2	Name: AIT LAHCEN Adam
Member 3	Name: BITAR Driss
Repository Link	`https://github.com/Adam-Ait-Lahcen/Base-de-donn-es`

1 Introduction

This lab focuses on identifying and stating functional dependencies (FDs) in the RLMS schema.

2 Requirements

Derive all functional dependencies (FDs) from the RLMS schema designed in Lab 1 and mapped to relations in Lab 2.

3 Team Roles

for this lab the work was carried out collaboratively, with each of the three team members initially developing an independent solution. These proposed solutions were then discussed, compared, and critically examined in order to identify points requiring clarification, correct inconsistencies, and retain the most relevant elements of each approach. Adam AIT LAHCEN primarily contributed to the revision of the proposed answers and the verification of their overall consistency, while Driss BITAR focused on integrating the different approaches into a coherent solution. Rayane BENYASSINE contributed further corrections and took primary responsibility for writing the report. However, these contributions should be understood as differences in emphasis rather than as strictly separate responsibilities. The final version resulted from a collective process in which all three members participated in reviewing, discussing, modifying, and approving the answers before submission.

4 Key Challenges and Their Resolution

The main difficulty was not simply identifying which attributes determine other attributes, but distinguishing between dependencies that follow from the given requirements and those that would require additional assumptions. A functional dependency cannot be established merely because it seems reasonable in practice; it must follow from the specifications or from an explicitly adopted assumption; One important discussion concerned whether the combination of a person's first name and last name determines their email address. We rejected this dependency. Two different people can share the same first and last names, and therefore these attributes do not uniquely identify a person. Furthermore, the uniqueness of institutional email addresses does not imply that email addresses can be derived from names. Even if email addresses can follow a naming convention, the possibility of collisions, different formats, or additional identifiers prevents us from concluding that the name determines the email address. Consequently, we retained pid as the primary identifier of Person and treated email $\rightarrow$ pid as valid only under the explicit assumption that institutional email addresses are unique. This distinction allowed us to separate the uniqueness of an identifier from the apparent relationship between descriptive attributes. Another point of discussion concerned the relationship between an employee's phone number and their identifier. Although each employee is associated with a phone number, the requirements do not specify that phone numbers must be unique across employees. Consequently, we did not consider phone $\rightarrow$ pid to be a valid functional dependency. Indeed, the fact that each employee has a phone number

does not imply that a phone number uniquely identifies an employee, since the same number could be shared by multiple employees. We could have introduced an additional assumption establishing the uniqueness of phone numbers, but this would have required us to explicitly adopt that assumption and include the corresponding dependency among the functional dependencies considered. Since we did neither, we retained only the dependencies justified by the given requirements, without introducing an additional uniqueness constraint on phone numbers. More generally, we distinguished functional dependencies from other types of constraints. For example, participation requirements and restrictions on the number of participants in a relationship do not necessarily establish functional dependencies. Similarly, a many-to-many relationship generally requires the combination of its participating identifiers to identify a tuple, rather than either identifier determining the other. This distinction was important when analyzing relationships such as Advises, Attached, Requires, and the funding relationships.

5 Justification of Conversion Choices

The conversion choices were primarily based on the relational schemas and their associated requirements. Primary identifiers determine the attributes describing the corresponding entity, while relationships with additional attributes were analyzed according to the entities involved and the cardinalities specified. For many-to-many relationships without additional attributes, the combination of the participating identifiers was retained as the candidate key. For relationships with attributes, such as Participates and Supplies, the dependencies were defined using the complete combination of attributes needed to determine those values. Weak entities, including Maintenance and CalibrationRecord, were identified through the combination of their owner's identifier and their respective partial key. Subtypes inherited the identifier of their parent entity, and their additional attributes were determined by that identifier. These choices preserve the structure of the original requirements without introducing dependencies that are not justified.